Although running the process above is fast, it can be made even faster. To do that, you'll use some tricks that are known in PostgreSQL to speed up the insert rate by taking away some of the database objects.

A note upfront though. These tricks speed up the Insert rate but add significant complexity to the process. For your needs try starting out with table copies that have all the database objects and run your scrubbing process to see if it's fast enough. If it's very slow, then proceed with these tricks, and understand you'll be putting in some more work to speed the process up.

Two of the tricks to speed up inserts are to remove database constraints and indexes on the destination table. After copying is completed you'll add them back. This can be a lot of direct manipulation and table modifications, which is a lot to learn! Don't worry, you'll conduct this work on your local PostgreSQL database and gradually build up your comfort with these techniques.

Open up the Rideshare database with psql. Drop the `users_copy` table since you're going to create it again. Earlier you created the `users_copy` table using the `INCLUDING ALL` keywords but you'll go in a different direction this time.

This time use the keywords `INCLUDING ALL EXCLUDING INDEXES`.

Run these statements:

sql/table_create_like_including_all_excluding_indexes.sql

```
DROP TABLE IF EXISTS users_copy;

CREATE TABLE users_copy (LIKE users INCLUDING ALL EXCLUDING INDEXES);
```

Explore the new version of `users_copy` with `\d users_copy`. This version has many of the database objects before except the table Indexes are missing and the Primary Key constraint on the `id` column is not there either.

With indexes on the table, each table write needs to be maintained in any Indexes covering columns being inserted. Without the indexes this index maintenance does not happen and inserts are faster. Later on you'll see how to run `CREATE INDEX` statements to recreate them all.

Run the "scrubbing_on_the_fly.sql" Insert statement again on the new `users_copy` table. If you've enabled `\timing` in psql, study the differences. Without indexes the newer version should be faster.

Next you'll consider what's missing in order to make sure a replacement table retains all of the original objects.